

Sistema de Vacunación - Entrega 2: Implementación del Modelo Funcional y Validación de Decisiones de Diseño

Diseño de Software - Grupo 5

Integrantes del grupo:

Benjamín Díaz

Diego Matus

Kurt Koserak

Víctor Galaz

Vicente Miranda

Benjamín Stuckrath

1. Introducción

Con el objetivo de profundizar y refinar las decisiones de diseño establecidas en la primera etapa de nuestro sistema de agenda y gestión de vacunación, en este segundo informe detallaremos la transición desde nuestro modelo conceptual hacia una primera implementación ejecutable.

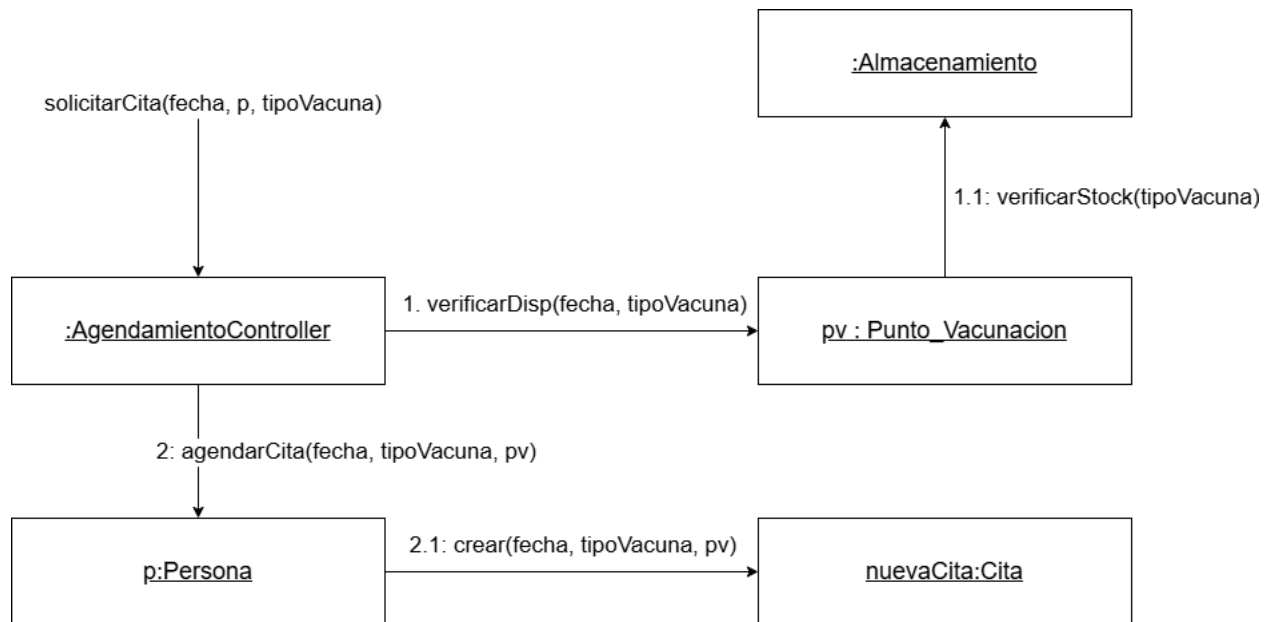
En este documento abordaremos de manera explícita la asignación de responsabilidades entre los distintos objetos del sistema mediante patrones GRASP. Al mismo tiempo, presentaremos un diagrama de clases refinado, incorporando métodos y comportamientos específicos que se desprenden directamente de las interacciones modeladas. Además detallaremos la incorporación estratégica de patrones de diseño software, abarcando las categorías creacionales, estructurales y de comportamiento para robustecer la arquitectura, junto con la definición de un esquema básico de seguridad. Se organizarán todos estos elementos técnicos dentro de un proceso de negocio estandarizado en notación BPMN. Si bien para esta etapa la interacción con el sistema se realizará mediante una interfaz intencionalmente simple, este enfoque nos permite mantener el énfasis estrictamente en la calidad del diseño interno, y en su correspondencia directa con el código desarrollado.

Al igual que en nuestra entrega anterior, buscaremos fundamentar nuestras decisiones metodológicas en cada uno de estos pasos para asegurar que la trazabilidad se mantenga intacta al pasar de la teoría a la práctica. De este modo, garantizamos la construcción de un software robusto y altamente cohesivo.

2. Diagramas de Comunicación

2.1 Creación de objetos

Explicación: El diagrama representa el flujo de creación de una nueva cita de vacunación. El patrón Controlador se utiliza en `AgendamientoController`, el cual actúa como punto de entrada para `solicitarCita()` y orquesta los mensajes, desacoplando la interfaz del usuario de la lógica del dominio. El patrón Experto se aplica en la validación de dos niveles: cuando el controlador consulta a Punto Vacunación sobre la disponibilidad, y cuando este a su vez delega el chequeo de inventario al Almacenamiento con `verificarStock()`. Finalmente, el patrón Creador se aplica al asignar a `p:Persona` la responsabilidad de instanciar `nuevaCita:Cita` a través de `crear()`. Esta decisión se justifica conceptualmente en el modelo de dominio, dado que la persona es la entidad responsable de registrar, agregar y contener su propio historial de citas.

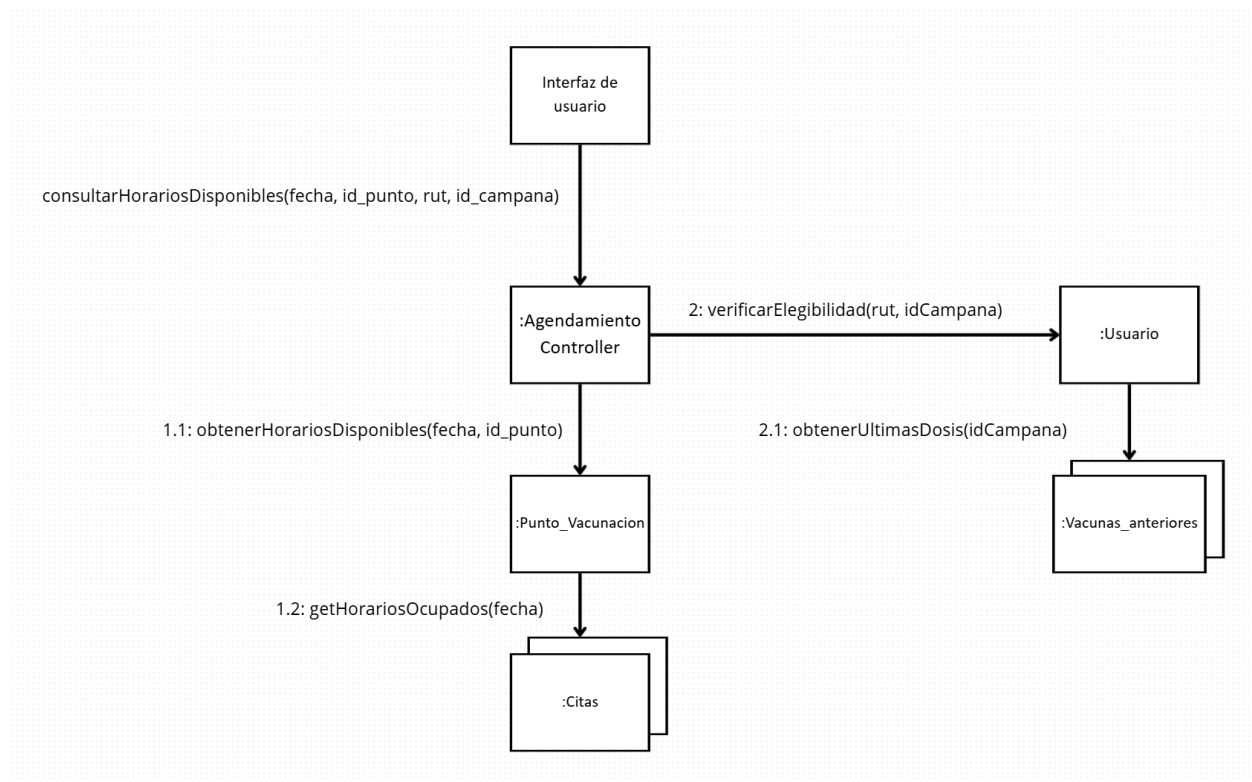


2.2 Consulta

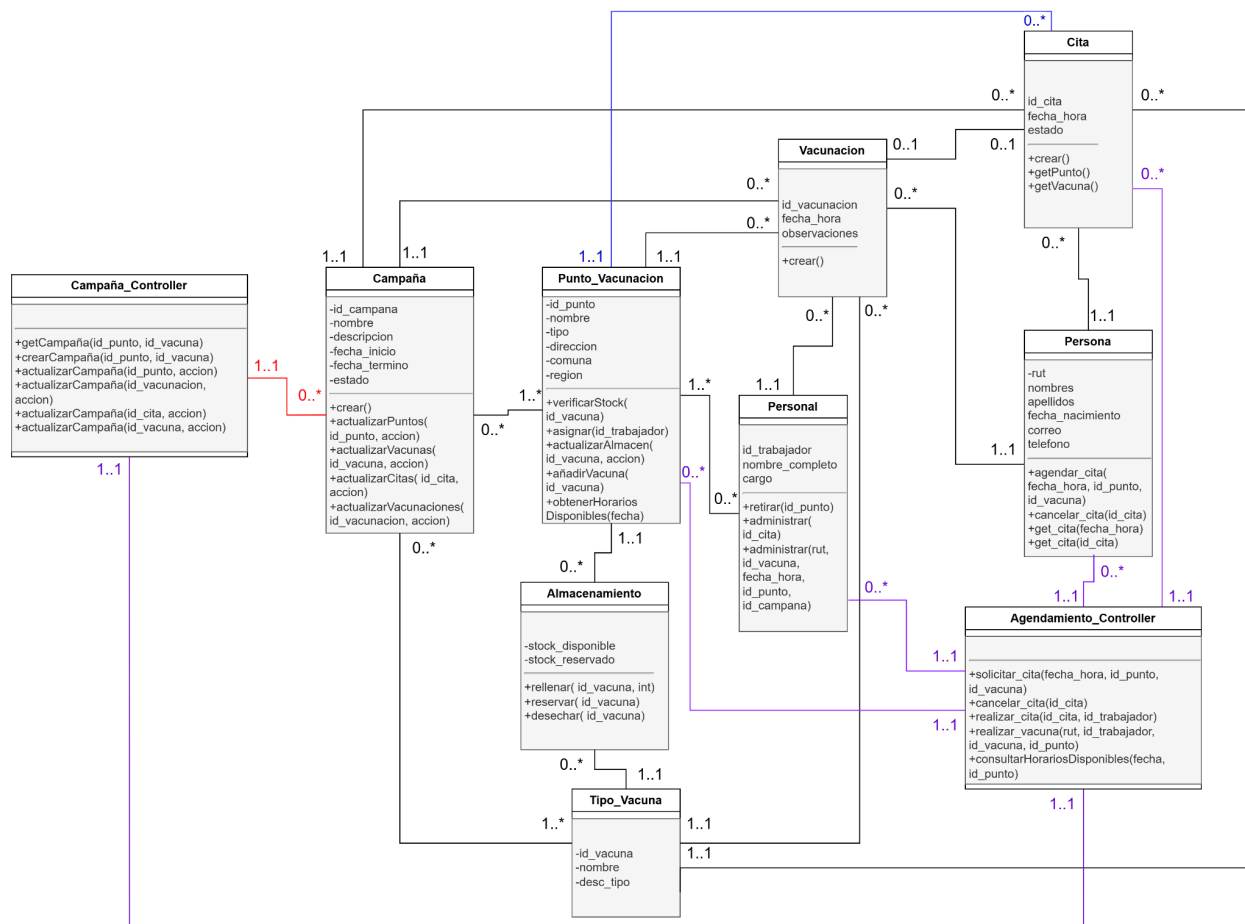
Explicación: El diagrama siguiente representa una consulta realizada dentro de la aplicación y como esta llama a los elementos de la aplicación con tal de entregar una respuesta a lo pedido, en este caso tenemos a un usuario que quiere agregar una cita para ir a un centro de vacunación a atenderse, pero para que la atención se lleve a cabo correctamente, deben haber vacunas en stock y también el usuario debe necesitar esta vacuna para empezar (Un usuario no se puede vacunar de lo mismo varias veces dentro de un plazo muy corto), El patrón de controlador se aplica en el agendamiento para delegar las solicitudes a otros segmentos para separar interfaz y lógica. En cuanto a patrones, experto se demuestra en :punto_vacunacion y :usuario, ya que estos elementos son los que tienen conocimiento sobre los datos de disponibilidad de horarios y los datos de vacunas anteriores respectivamente.

Bajo acoplamiento: Elementos como el usuario y el punto de vacunación son usados para datos totalmente distintos, pero el controlador de agendamiento se encarga de pedir estos datos para completar la consulta

Alta cohesión: Cada uno de los elementos se encargan de tratar con sus propios datos contenidos dentro del mismo objeto, mientras que el punto de vacunación se encarga de guardar los horarios, usuario solo se encarga de guardar su agenda médica de vacunas anteriores.



3. Diagrama de Clases Refinado



Observaciones sobre decisiones de diseño:

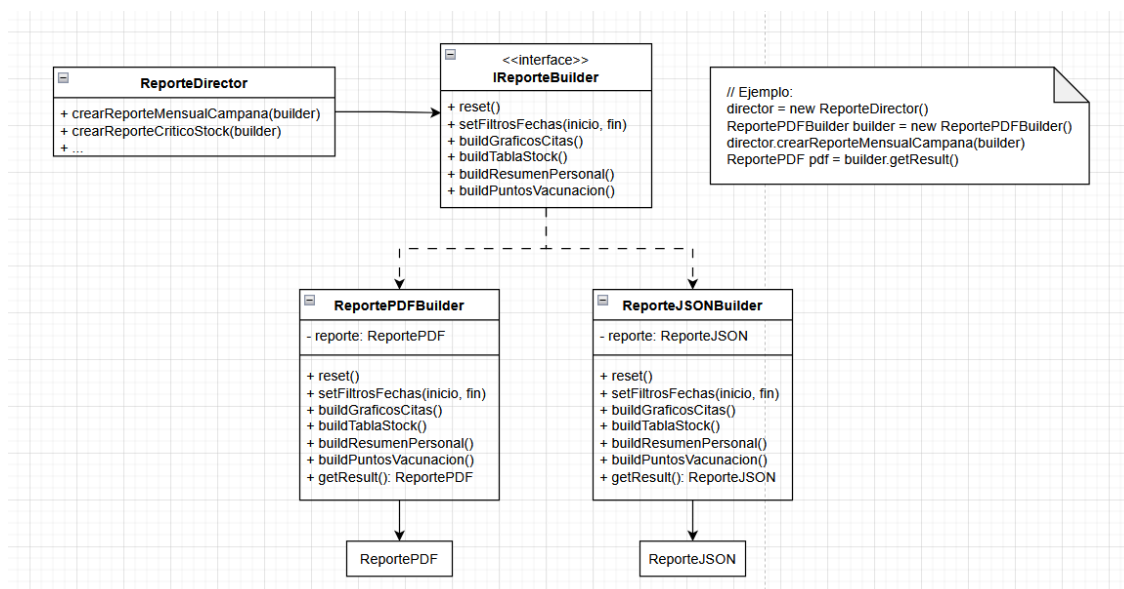
- 1) Sobre creación de otras clases: Campaña_Controller es el encargado de crear Campañas, Punto_Vacunación crea Almacenamientos para las vacunas nuevas que recibe, Personal crea Vacunaciones al ser el encargado de aplicarlas, y Persona crea sus respectivas citas, aunque es Agendamento_Controller quien recibe la instrucción inicial.
- 2) Sobre Campaña: Recibe los comandos a ejecutar a partir de Campaña_Controller, y desde Campaña se comunica con la clase correspondiente para añadirla o excluirla de la campaña.
- 3) Sobre Agendamento_Controller: A pesar de tener diversas funcionalidades a través de sus métodos, estos son delegados a otras clases para mantener un bajo acoplamiento (por ejemplo, solicitar_cita delega sus funcionalidades a Persona y a Campaña_Controller)
- 4) Mensajes que se pueden mandar en el diagrama: Creación y modificación de Campañas y de las clases a las que está relacionada, modificación del Almacenamiento de un Punto_Vacunacion, creación y administración de Vacunaciones, creación y eliminación de Citas, consulta de horarios disponibles en un Punto_Vacunación, consultar historial de Vacunaciones de una Persona, entre otros.

4. Diagramas de patrones de diseño

4.1 Diagrama de patrón creacional (Builder):

Los administradores necesitan informes de las campañas y de gestión con estructuras variables, por ejemplo, algunos llevan gráficos de barras del stock disponible o de progreso de citas, otros tablas de asistencia por comuna/región, otros la adherencia de puntos de vacunación a la campaña, etc. Crear estos informes mediante un constructor tradicional generaría un método muy grande lleno de parámetros opcionales, u obligaría a crear subclases rígidas para cada combinación posible de reporte.

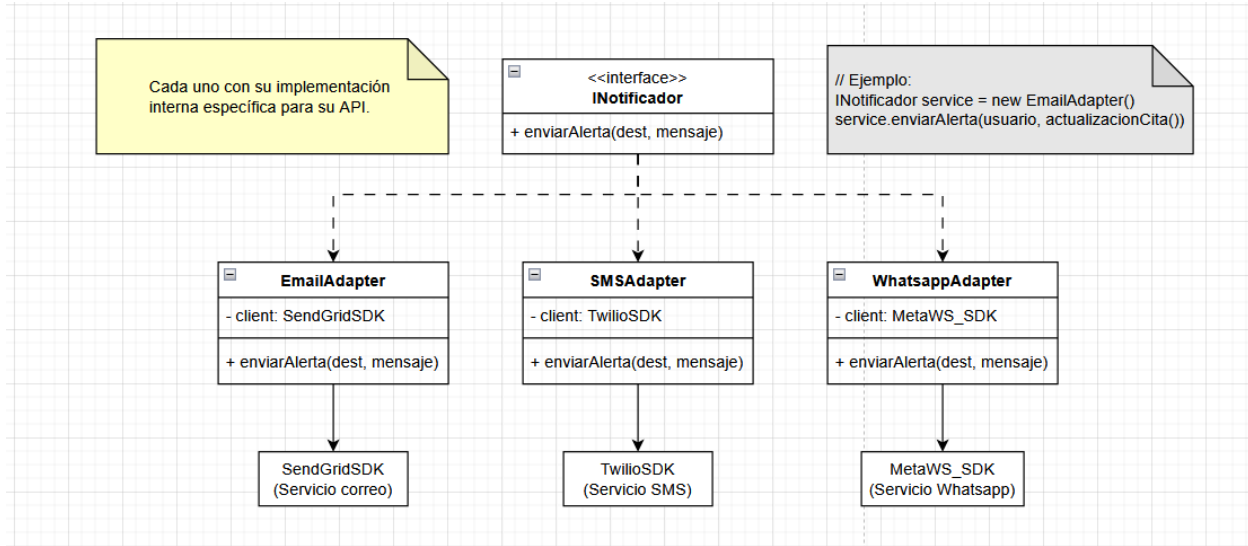
Con este patrón, se desacopla completamente la lógica de recopilación de datos de la representación visual del informe. Permite que un objeto director ordene la construcción de los reportes paso a paso (buildGrafico(), buildTabla(), buildPuntosVacunacion(), etc), obteniendo un producto final más flexible.



4.2 Diagrama de patrón estructural (Adapter):

El sistema debe entregar alertas por múltiples canales (email, sms, Whatsapp, etc) interactuando con proveedores externos. Integrar estos proveedores de forma directa en el flujo de la reserva de citas crearía una dependencia rígida. Y si por ejemplo, el proveedor cambia sus métodos, actualiza su API, o se decide cambiar de proveedor, se rompería la lógica de negocio.

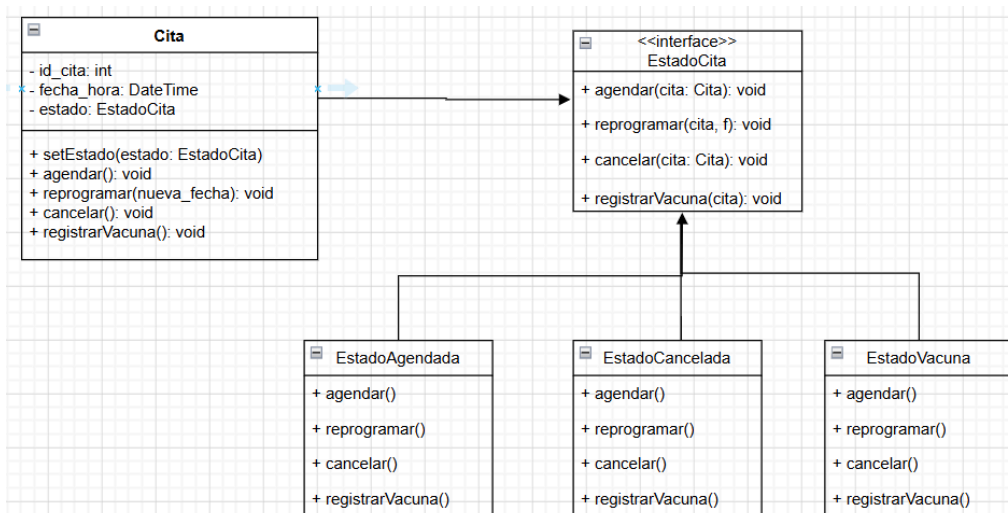
El patrón Adapter proporciona un bajo acoplamiento, el flujo del software solo conoce una interfaz limpia y genérica 'enviarAlerta()', y los cambios de proveedores externos quedan encapsulados exclusivamente dentro de sus respectivos adaptadores, haciendo que el sistema no se vea gravemente perjudicado a cambios.



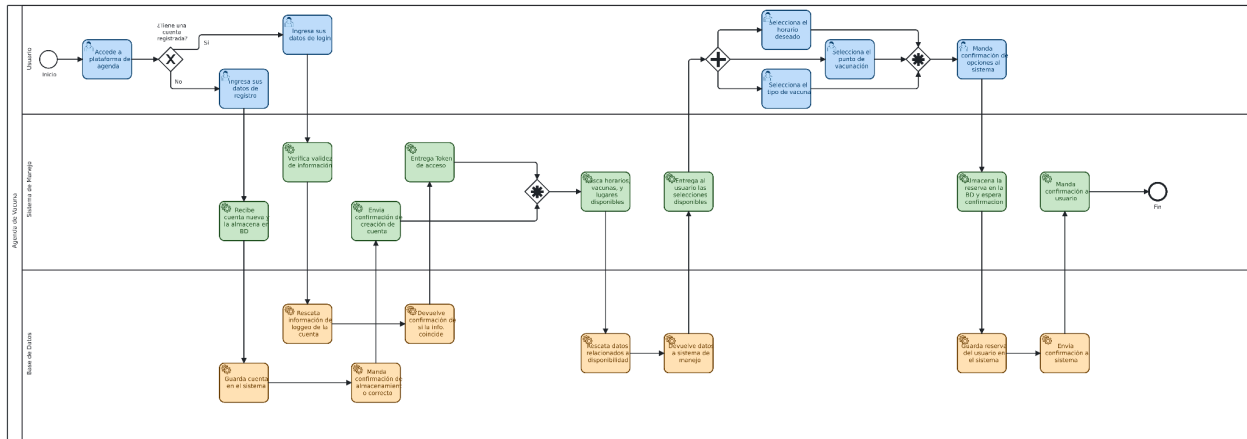
4.3 Diagrama de patrón de comportamiento (State):

Para reservar citas, el ciclo de vida de una Cita transiciona por múltiples estados válidos, como “agendada”, “cancelada” o “resuelta”. Las reglas de negocio restringen qué acciones se pueden tomar en cada fase. Por ejemplo, una cita reprogramada o cancelada no puede ser procesada del mismo modo que una cita activa. Resolver esto mediante condicionales (if o else) dentro de la entidad Cita rompería el principio de responsabilidad única y generaría un código rígido abierto a errores de consistencia.

Con el patrón State, se encapsulan los comportamientos específicos de cada estado de la cita en clases independientes que implementan una interfaz común. La clase Cita delega dinámicamente los procesos dependiendo del estado que tenga asignado en tiempo de ejecución. Esto garantiza que el diseño del sistema entre en estados inconsistentes y permite añadir con facilidad nuevos estados futuros en distintas campañas sin alterar las clases existentes.



5. Diagrama de proceso BPMN



(Imagen de mayor calidad subida a repositorio)

6. Descripción de seguridad básica

1. Control de acceso fino

Cómo dividiremos según el rol de la persona por persona:

Usuario normal: Identificado por su cuenta, cada usuario normal debe poder acceder y modificar sus datos personales, también debe ser capaz de consultar sus propias consultas y ser capaz de editar los datos de esta.

Personal de la salud: Los funcionarios tienen acceso a las consultas pedidas por los usuarios para confirmarlas y terminarlas una vez se realice la vacuna, cada funcionario solo puede modificar las consultas que corresponden al punto de vacunación al que pertenecen.

Administrador: Solo existe uno o muy pocos, tienen acceso a todos los datos relacionados a cuántas vacunas se realizaron, qué centros fueron los más visitados, etc. No tienen acceso a los datos personales de cada usuario, pero usan los datos estadísticos para realizar reportes sobre la efectividad de la campaña con tal de enviar los números al gobierno.

2. Protección de datos sensibles

Para corroborar que los datos están resguardados de forma segura, sólo se recolectará la información necesaria de cada persona y cada uno de estos datos será guardado en bases de datos externas que nos aseguren el buen cuidado de la información.

3. Registro de auditoría

Para llevar un buen seguimiento en el cambio de los datos, existirá logs donde se puedan acceder a las operaciones que se realizaron durante la campaña con tal que se puedan realizar correcciones o que el administrador pueda detectar anomalías, cada operación incluye quien hizo el cambio, cuando lo realizó y la dirección IP de donde se envió.

4. Comunicación segura entre componentes

Las aplicaciones finales deberán apegarse a los estándares de la industria con tal de mantener los datos en un lugar seguro, mientras que los componentes se apegan a una sola función y se comunican con el resto para realizar sus funciones.

5. Manejo de sesiones y revocación

Cada usuario solo podrá tener una sesión asociada a su correo o RUT, la cuenta se mantendrá en una base de datos siempre y cuando la cuenta se siga usando cada cierta cantidad de años (~ 5 a 10 años de espera antes de que la cuenta se elimine automáticamente), también usaremos Two-Factor Authentication para confirmar que las sesiones sean creadas por las personas correctas.

Si no se tiene una cuenta, no se podrá acceder a ninguna de las funcionalidades de la aplicación y una persona.

6. Prevención de ataques

Tendremos validación de usuarios, conexiones y archivos, usaremos los avances actuales para notar cualquier tipo de anomalía en el uso de nuestra aplicación, esto incluye usar herramientas para el análisis en el tráfico de red dentro de nuestra aplicación.

7. Conclusiones

Este entregable nos permite consolidar la transición entre la arquitectura conceptual abstracta planteada originalmente y una solución de software tangible y ejecutable.

La aplicación rigurosa de los patrones de asignación de responsabilidades GRASP y de los patrones de diseño (en este caso implementados el Builder, Adapter y State) ayudaron a que el equipo logre demostrar que un diseño de software de alta calidad es el camino directo a la construcción de un producto viable y no sólo de un artefacto aislado.

Un hito clave para este proceso fue la utilización e implementación de BPMN, permitiendo que las reglas del negocio del sistema de vacunación se ejecuten de manera coordinada entre los objetos del dominio bajo un esquema de seguridad básico pero robusto.

En definitiva, este entregable valida que el acoplamiento controlado, la alta cohesión y la separación de interfaces son atributos indispensables para garantizar la mantenibilidad y extensibilidad de una plataforma crítica de salud pública, la cuál es muy propensa a futuros cambios operacionales.